

PAKS: Predictive Adaptive Kubernetes Scaling versus Reactive HPA

Pooja | 25120921 | MSc Cloud Computing, National College of Ireland

Abstract

Cloud applications on Kubernetes face highly variable workloads. The Horizontal Pod Autoscaler (HPA) reacts after metrics cross targets, so control delay under bursts can under- or over-provision. This study asks whether predictive workload forecasting with adaptive Kubernetes scaling (PAKS) improves resource management versus reactive HPA. The artefact trains a NumPy LSTM on disclosed Google Cluster Trace and Alibaba samples (TensorFlow fail-closed where unavailable), implements an adaptive scaler via the Kubernetes API, and evaluates on Free-Tier AWS EC2 single-node k3s with S3 and CloudWatch (destroy-after). On TRACE slices, LSTM MAE is worse than persistence on held-out GCT 2011 job windows (~ 0.102 versus ~ 0.048) and on an Alibaba RANGE CPU% sample (~ 3.435 versus ~ 2.803). On live AWS k3s (formal_k8s_live_aws.json; $n=16$ steps; max 3 replicas), mean scaling latency is ~ 0.40 s for HPA versus ~ 0.31 s for PAKS, with final-3 rounds non-monotone; cost and some SLA fields remain partly SIMULATED. Confirmatory superiority of PAKS over HPA is not claimed at lite n. The study delivers a reproducible PAKS-versus-HPA harness with TRACE/LIVE/SIMULATED tagging; full dumps, larger live campaigns, and billing-linked cost remain residuals.

Introduction

Kubernetes is the dominant orchestrator for containerised cloud services. HPA reacts to CPU, memory, or custom metrics to add or remove pods. Predictive autoscaling literature trains forecasters such as LSTM models on cluster traces including Google Cluster Trace and Alibaba Cluster Trace, then feeds predictions into scaling policies to reduce SLA violations and waste. Workload bursts make purely reactive control costly: lag induces SLA risk, while aggressive headroom wastes money. Showing whether a predictive adaptive controller improves on HPA—on public traces

and on AWS-hosted Kubernetes—matters to platform engineers and to researchers comparing control policies under explicit evidence tags. Prior proxy work framed a different NimbusGuard and synthetic niche; formal CA2 binds HPA as the baseline and GCT/Alibaba plus AWS Kubernetes as the method.

The study asks whether predictive workload forecasting combined with adaptive Kubernetes scaling (PAKS) can improve cloud resource management under dynamic workloads versus traditional reactive Kubernetes HPA. The work sets out to preprocess public traces and train a workload predictor (LSTM / TensorFlow as named in CA2 resources), with success defined by TRACE MAE/RMSE tables on disclosed GCT and Alibaba samples with provenance and with TensorFlow fail-closed rather than silently substituted on the formal path. It builds an adaptive scaling engine that acts via the Kubernetes API rather than HPA-default alone, exercising Scale PATCH paths in dry-run and/or live modes with logged operations. Evaluation on Kubernetes hosted on AWS—EC2, S3, and CloudWatch—requires live cluster evidence JSON with `aws_deployed` and destroy verification. Comparison of PAKS versus reactive HPA across workload intensities uses the formal metric suite with side-by-side rows tagged LIVE, TRACE, or SIMULATED and no invented billing figures.

The design is quantitative and experimental: disclose and hash-verify trace samples; train NumPy LSTM (TensorFlow requested, ImportError fail-closed on the eval host); run adaptive versus reactive policies against Kubernetes Scale; measure prediction error on TRACE holds and scaling latency on LIVE k3s; and retain a labelled proxy simulator only as non-binding. Final-3 live packs repeat lite destroy-after rounds. The live cluster is single-node Free-Tier k3s with small step counts and max replica caps—not a full multi-AZ stress suite. Full dumps were not fetched. Cost, SLA, and response-throughput fields may be simulated or assumed. LSTM underperforms persistence on current TRACE slices, so results are directional at lite n. Assumed USD-per-pod-hour is used where billing export is absent; synthetic cyclical loads without GCT/Alibaba-derived behaviour remain proxy-only; destroy-after prevents leftover AWS spend; and HPA configuration in the harness is treated as a fair reactive peer for the lite demo deployment.

Section 2 reviews reactive HPA and predictive autoscaling, treating NimbusGuard as related work only. Section 3 describes HPA versus PAKS control. Section 4 covers trace loaders, LSTM, scaler,

and the quarantined proxy. Section 5 reports TRACE prediction metrics and LIVE scaling latency with provenance tags. Section 6 concludes and lists optional beyond-floor residuals.